ERHVERVSAKADEMI AARHUS

# Machine Learning.

Vibe coding rules.

*Sila. January 25th, 2026.*

Specs: Make goals, constraints, and acceptance criteria explicit before agents start building.

Human control: Keep people where decisions carry real risk and responsibility.

Agentic Engineering: Give autonomous agents clear roles, boundaries, tools, and coordination.

Harness Engineering: Use tests, rules, feedback loops, and guardrails to catch and correct failures.

Architecture & Security: Set boundaries before generated code reaches critical systems.

Governance & Quality: Define what AI-generated software must prove before it reaches production.

From Vibe to AI-driven Software Engineering:

https://vibekode.it/blog/ai-driven-software-engineering-vibe-coding/

## Med flere ord:

1. Specs (Udvidelse)

- **Tilføjelse: "Executable Specifications" (Eksplicitte eksempler).** AI-agenter forstår konkrete eksempler (f.eks. Given-When-Then formater eller input-output par) markant bedre end abstrakte målsætninger.

- **Tilføjelse: Arkitektoniske rammer.** Agenten skal kende kodestandarden (f.eks. "Brug Functional Programming", "Følg SOLID-principperne" eller "Brug dette specifikke design pattern") *før* den begynder, ellers gætter den sig til stilen.

2. Human control (Udvidelse)

- **Tilføjelse: Fra "Human-in-the-loop" til "Human-on-the-loop".** Definer præcis *hvornår* mennesket skal godkende. I stedet for at godkende hver linje kode (hvilket dræber hastigheden), bør kontrollen ligge ved **Arkitektur-review** (inden start) og **Pull Request / Code Review** (inden integration).
- **Tilføjelse: Intent vs. Output.** Menneskets rolle skifter fra at skrive koden til at validere, om agentens *forståelse* af opgaven matcher det reelle forretningsbehov.

3. Agentic Engineering (Udvidelse)

- **Tilføjelse: State Management & Hukommelse.** Agenter fejler ofte på lange eller komplekse opgaver, fordi de mister kontekst ("context drift"). Der skal være en klar struktur for, hvordan agenter deler viden (f.eks. via en fælles "Blackboard" arkitektur eller git-branches).
- **Tilføjelse: Deterministiske værktøjer.** Agenter skal have adgang til værktøjer, der giver entydige svar (f.eks. AST-parsere, linters og sandboxed CLI'er) frem for kun at forlade sig på sandsynlighedsbaseret LLM-logik.

4. Harness Engineering (Udvidelse)

- **Tilføjelse: "Self-Healing" loops (Automatisk fejlretning).** Hvis en test fejler, skal test-outputtet (f.eks. en stack trace) automatisk fødes tilbage til agenten, så den kan rette sin egen fejl uden menneskelig indblanding.
- **Tilføjelse: Evaluering af test-kvalitet (Mutation Testing).** Agenter kan godt finde på at skrive tests, der altid består (f.eks. uden reelle asserts) for at opfylde et mål om testdækning. Dit harness skal validere, at agentens tests faktisk er robuste.

5. Architecture & Security (Udvidelse)

- **Tilføjelse: Sandboxing under udvikling.** Agenter skal køre og teste deres kode i isolerede miljøer (f.eks. Docker-containere, kortlivede cloud-miljøer) uden adgang til interne netværk eller rigtige API-nøgler.
- **Tilføjelse: Statisk sårbarhedsscanning.** AI har en tendens til at genbruge kendte mønstre, som kan introducere klassiske sårbarheder (f.eks. SQL-injections eller hardcoded credentials). Automatiserede sikkerhedsscannere skal køre på alt agent-genereret kode.

6. Governance & Quality (Udvidelse)

- **Tilføjelse: AI-proveniens (Sporbarhed).** Det skal være tydeligt i dit versionsstyringssystem (Git), hvilke dele af koden der er skrevet af en agent, hvilken prompt/specifikation den blev skabt ud fra, og hvem der har godkendt den.
- **Tilføjelse: Vedligeholdelsesomkostninger (Technical Debt).** AI kan generere enorme mængder kode lynhurtigt. Governance skal sikre, at koden ikke bare virker her og nu, medmindre den også er læsbar og vedligeholdelig for mennesker i fremtiden.